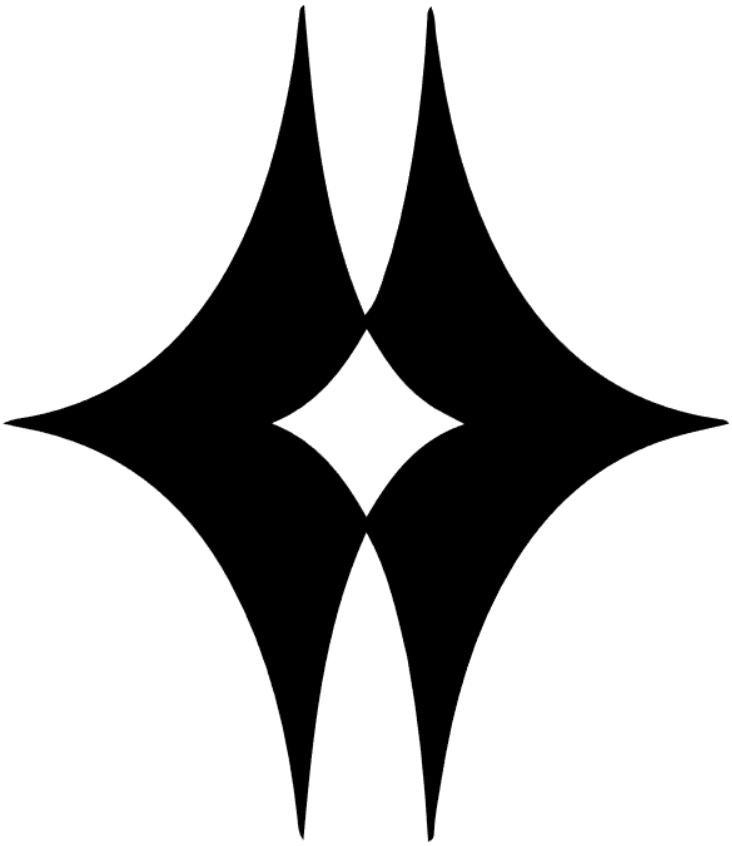
PROYECTO INTERMODULAR DAM



ASTRAÏS

Olga, Samuel, Manuel, Elías

ÍNDICE

1 - DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROYECTO 3
1.1) Problema real y contexto 3
1.2) Propuesta de valor 3
1.3) Alcance 4
2 - VALORACIÓN DE MERCADO 5
2.1) Habitica 5
2.2) Skillion/Motion/Eterya (apps de motivación gamificada) 5
3 - STACK TECNOLÓGICO ELEGIDO 6
3.1) Backend (Servidor) 6
3.2) App móvil (Android) 6
3.3) App web 7
4 - OBJETIVOS DEL PROYECTO 7
4.1) Objetivo general 7
4.2) Objetivos específicos 7
5 - REQUISITOS DEL SISTEMA 8
5.1) Requisitos funcionales 8
5.2) Requisitos no funcionales 9
5.3) Requisitos de interfaz 9
6 - CASOS DE USO 10
Caso 1: Inicio de sesión normal 10
Caso 2: Creación de tareas 10
Caso 3: Completar tarea y obtener recompensas 10
Caso 4: Comprar cosmético en tienda 11
Caso 5: Crear grupo e invitar miembros 11
Caso 6: Minijuego con recompensa 12
Caso 7: Admin gestiona catálogo de tienda 12
7 - MODELO Y DISEÑO BD 13
7.1) Entidades principales 13
8 - LÓGICA DE NEGOCIO MÍNIMA 13

1 - DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROYECTO

1.1) Problema real y contexto

Uno de los mayores obstáculos para mantener la productividad personal es la falta de constancia. Aunque muchas personas tratan de usar sistemas de organización (listas, calendarios, apps), suelen abandonarlos en poco tiempo debido a:

- La falta de motivación inmediata, porque no perciben recompensas visibles tras completar actividades.
- La sensación de monotonía por rutinas repetitivas.
- La ausencia de refuerzo positivo visible que refuerce el avance personal.
- La dificultad para compartir objetivos de forma colaborativa.

Astraïs propone resolver este problema mediante la gamificación de hábitos y tareas cotidianas, transformando la rutina en una experiencia interactiva inspirada en videojuegos: progresión por niveles, desbloqueo de logros, moneda virtual, tienda de recompensas, minijuegos integrados y opciones de personalización. De este modo, convierte lo ordinario en algo más especial para fomentar la constancia a largo plazo.

1.2) Propuesta de valor

Astraïs convierte la productividad en un "progreso de personaje". Más allá de gestionar tareas, el usuario puede notar crecimiento continuo: sube de nivel, desbloquea cosméticos, personaliza su avatar, adopta mascotas y accede a minijuegos. Además, está diseñado para evitar el famoso P2W (*pay to win*): el avance (nivel, logros) se obtiene mediante esfuerzo y constancia, no mediante pagos.

Pilares fundamentales del diseño:

- **Motivación y refuerzo**: Sistema de recompensas inmediatas mediante Ludiones (moneda virtual), puntos de experiencia (XP) y logros significativos.
- **Progreso tangible y visual**: Avance medible a través de niveles, rachas de cumplimiento y opciones de personalización.

- **Conexión social:** Interacción mediante amigos, grupos colaborativos, tareas compartidas y retos colectivos.
- **Variedad:** Minijuegos y eventos temporales que rompen la repetición diaria, manteniendo la aplicación activa sin sacrificar el foco en el objetivo real (productividad personal).
- **Utilidad real:** Base de herramientas prácticas (calendario, gestión de tareas y planificación).

1.3) Alcance

Alcance inicial:

- **Gestión de tareas:** Sistema completo para tareas individuales y grupales, con asignación de responsabilidades, plazos y seguimiento.
- **Colaboración en equipo:** Grupos con gestión de miembros, roles diferenciados (líder, colaborador) y tareas compartidas.
- **Economía virtual:** Moneda interna (Ludiones) obtenida mediante el cumplimiento de tareas.
- **Personalización:** Tienda integrada para adquirir cosméticos, personalización del avatar (rasgos, vestimenta) y sistema de mascotas.
- **Progresión:** Sistema de niveles basado en XP acumulada, rachas de cumplimiento y logros.
- **Interacción social:** Sistema de amigos e invitaciones a grupos y compartir tareas.
- **Entretenimiento:** Minijuegos breves y eventos puntuales cuyo acceso y recompensas están directamente vinculados al rendimiento productivo real.
- **Plataformas:** App móvil (Android) + App Web de apoyo/administración/consulta.

Fuera de alcance inicial (para fases futuras):

- Inteligencia artificial para planificación autónoma, sugerencia de prioridades o distribución inteligente de tareas.
- Integración con Google Calendar/Outlook/Apple Calendar.
- Marketplace de skins creadas por usuarios.

2 - VALORACIÓN DE MERCADO

2.1) Habitica

Pros: gamificación completa y probada, comunidad activa de millones de usuarios, misiones cooperativas (no son tareas).
Contras: interfaz visual sobrecargada con curva de aprendizaje pronunciada para nuevos usuarios, estética retro (puede ser poco atractiva para personas que no estén acostumbradas), personalización estática sin evolución (cosméticos planos, sin mascotas interactivas ni sistema de colección con rareza), más enfocada en gamificación que utilidad.

Oportunidad para Astraïs: experiencia visual limpia y moderna (Jetpack Compose/Tailwind CSS), minijuegos que no comprometen la productividad, mascotas y cosméticos coleccionables con niveles de rareza.

2.2) Skillion/Motion/Eterya (apps de motivación gamificada)

Pros: diseño visual minimalista, gamificación ligera que facilita la entrada a la app mediante hábitos y rutinas simples.
Contras: economía virtual superficial, interacción social limitada a compartir logros sin colaboración real, experiencia repetitiva, más enfocadas en gamificación que utilidad.

Oportunidad para Astraïs: ser un híbrido entre productividad funcional y juego social. Se conserva la elegancia visual de estas apps, pero se añade profundidad mediante economía interna (Ludiones + tienda + transacciones), colaboración en equipos con roles definidos y minijuegos breves.

3 - STACK TECNOLÓGICO ELEGIDO

3.1) Backend (Servidor)

Docker + PostgreSQL + Ktor + Exposed

- **Ktor (Kotlin)**: Ktor deja hacer servidores de alto rendimiento ligeros y modulares pero con la flexibilidad de un lenguaje de alto nivel como kotlin. También deja convertir el código JVM al que se transpila kotlin en código máquina para mayor rendimiento y aprovechamiento de recursos.
- **Exposed**: Es un framework ORM con bastante flexibilidad en la que se pueden trabajar con varios tipos de base de datos relacionales (Postgres, MySQL, SQLite, Oracle, ...) usando el mismo código, lo cual brinda una flexibilidad considerable si se decidiera usar otra base de datos en cualquier punto.
- **Postgres**: Sistema de gestión de bases de datos relacionales orientadas a objetos avanzado y de código abierto, bastante robusto y con tipos de datos más avanzados que otras (como el tipo JSON o JSONB) que permite la creación de extensiones.
- **Docker**: Usaremos docker para el backend por su portabilidad, fácil escalación y por lo ligero que es.

3.2) App móvil (Android)

Kotlin + Android Studio + Jetpack Compose

- **Kotlin:** lenguaje moderno, conciso y seguro contra errores nulos, ideal para código mantenible y robusto en Android.
- **Jetpack Compose:** UI declarativa que acelera el desarrollo de pantallas complejas como tienda, logros, avatar o listas dinámicas de grupos/amigos. Componentes reutilizables y reactivos que se actualizan automáticamente ante cambios de estado.
- **Android Studio:** herramientas nativas para preview en tiempo real, debugging visual y profiling de rendimiento.
- **Arquitectura:** Compose encaja perfectamente con MVVM + StateFlow, separando lógica de negocio de la interfaz para facilitar testing y escalabilidad.

3.3) App web

React + TypeScript + Vite + Tailwind + Firebase

- **React + TypeScript:** Arquitectura modular que facilita mantenimiento y escalabilidad, tipado estático que previene errores en tiempo de desarrollo, mejora autocompletado y refactorización segura.
- **Vite:** configuración simple, su principal ventaja es la velocidad que ofrece desde el primer segundo. Tiene con soporte nativo para TypeScript sin necesidad de plugins adicionales.
- **Tailwind:** Diseño unificado con los elementos predefinidos (colores, tipografía), lo que permite un desarrollo rápido y responsivo.
- **Firebase:** Hosting web.

4 - OBJETIVOS DEL PROYECTO

4.1) Objetivo general

Desarrollar una solución multiplataforma (app móvil Android + web) que mejore la constancia de los usuarios en la gestión de tareas mediante gamificación, motivación progresiva, recompensas tangibles y colaboración social.

4.2) Objetivos específicos

- Permitir crear, editar y completar tareas personales y grupales con asignación de fechas, prioridades y seguimiento visual del progreso.
- Implementar XP por tareas completadas y Ludiones por rachas y logros, con conversión automática y visualización en tiempo real.
- Implementar niveles, logros y tienda con cosméticos coleccionables.
- Implementar perfiles con avatar y mascota.
- Gestionar amigos, crear grupos (hasta 10 miembros), enviar invitaciones y asignar tareas compartidas con notificaciones.
- Implementar minijuegos conectados a progresión.
- Garantizar autenticación segura + roles (admin/usuario/invitado).
- Proveer un backend desplegable con Docker para evaluación.

5 - REQUISITOS DEL SISTEMA

5.1) Requisitos funcionales

- **Autenticación y cuentas**

RF-01: Registro de usuario con validación de correo.
RF-02: Inicio/cierre de sesión seguros con persistencia controlada.
RF-03: Recuperación y cambio de contraseña mediante enlace temporal.
RF-04: Edición completa de perfil: nombre, biografía, avatar y mascota.
RF-05: Gestión de roles con permisos diferenciados (admin, usuario, invitado).

- **Tareas y calendario**

RF-10: Crear, editar y eliminar tareas con estado dinámico.
RF-11: Marcar tareas como completadas/pendientes.
RF-12: Asignar fecha límite, prioridad (baja/media/alta), etiquetas y recordatorios.
RF-13: Vista calendario (día/semana/mes).
RF-14: Crear y gestionar tareas grupales con asignación específica a miembros.

- **Gamificación**

RF-20: Sistema de XP y niveles.
RF-21: Otorgamiento de Ludiones por tareas completadas, retos y rachas.
RF-22: Logros con desbloqueo automático y exhibición en perfil.
RF-23: Seguimiento de rachas con recompensas (opcional).

- **Economía virtual y tienda**

RF-30: Monedero con saldo visible de Ludiones e historial de transacciones.
RF-31: Tienda con catálogo organizado de cosméticos (avatar, mascota) por categoría y rareza.
RF-32: Compra de artículos con Ludiones y actualización instantánea del inventario.
RF-33: Inventario personal para gestionar y equipar cosméticos adquiridos.

- **Social**

RF-40: Envío, aceptación y rechazo de solicitudes de amistad con notificaciones.
RF-41: Creación de grupos, invitación de miembros y gestión de roles internos (líder, colaborador).
RF-42: Chat en grupo (excluido del alcance inicial).
RF-43: Invitación a minijuegos mediante sistema de eventos o solicitudes directas.

- Minijuegos

RF-50: Acceso a lista de minijuegos.
RF-51: Sesión de minijuego vinculada a recompensa.
RF-52: Registro de resultados y estadísticas básicas.

- Administración

RF-60: Panel admin web / Android para: gestionar catálogo de tienda, moderar usuarios (ban), consultar métricas básicas y configurar parámetros de recompensas.

5.2) Requisitos no funcionales

RNF-01: Seguridad: contraseñas hasheadas (BCrypt), JWT, HTTPS en despliegue.
RNF-02: Rendimiento: endpoints críticos < 300 ms en local/entorno docente.
RNF-03: Escalabilidad: Arquitectura dockerizada con separación clara app/DB.
RNF-04: Mantenibilidad: Estructura por capas (domain/service/repository), documentación técnica y cobertura mínima de tests unitarios.
RNF-05: Disponibilidad: Persistencia de datos ante reinicios; respaldo básico de base de datos.
RNF-06: Accesibilidad: contraste, tamaños, compatibilidad con lectores de pantalla en Android.
RNF-07: Privacidad: Mínima recolección de datos; control de visibilidad del perfil (público/amigos/privado).
RNF-08: Observabilidad: Logs estructurados (JSON) y trazas básicas.

5.3) Requisitos de interfaz

RUI-01: Navegación por tabs (Inicio/Grupos/Juegos/Perfil), menú lateral en web.
RUI-02: Feedback visual con microanimaciones al obtener XP/Ludiones o completar acciones clave.
RUI-03: Diseño responsive en web.
RUI-04: Estados vacíos (sin tareas, sin grupos, sin amigos).
RUI-05: Modo oscuro.

6 - CASOS DE USO

Caso 1: Inicio de sesión normal

Accionador	Usuario
Condiciones	Hay que tener cuenta registrada (invitado o usuario normal)
Pasos a seguir	- Se introducen las credenciales - El backend valida al usuario - El backend devuelve los JWT junto a algún dato relevante del usuario (rol, insignias, avatares, etc...)

Caso 2: Creación de tareas

Accionador	Usuario
Condiciones	No hay condiciones
Pasos a seguir	- Se escribe el título, fecha, prioridad y tipo - En el caso de ser una tarea grupal: Se tendrá que seleccionar los usuarios asignados o si es para todos. - Se guarda la tarea y esta aparece en el calendario.

Caso 3: Completar tarea y obtener recompensas

Accionador	Usuario
Condiciones	No hay condiciones
Pasos a seguir	- El usuario selecciona la tarea y la marca como completada. - El backend calcula la recompensa a dar teniendo en cuenta los límites diarios o bonos.

	- Si se subiera de nivel: Se actualiza el nivel. - Se verifica si se completa algún logro con esta tarea. - Se avisa al cliente de todo lo que consiguiera con la tarea.

Caso 4: Comprar cosmético en tienda

Accionador	Usuario
Condiciones	Tiene que tener el saldo suficiente
Pasos a seguir	- El backend recibe la petición de compra y hace validaciones para ver si se puede comprar por si acaso el cliente estuviera manipulado de forma externa. - Si se pudiera realizar: Archiva la compra en la base de datos, quita los ludiones de la transacción al jugador, le da al cliente su objeto y avisa al cliente de que la transacción se hizo. - Si no se pudiera realizar: Envía un mensaje de error al cliente

Caso 5: Crear grupo e invitar miembros

Accionador	Usuario
Condiciones	Sesión iniciada con una cuenta de 'usuario'.
Pasos a seguir	Creación de grupo: - Al recibir la petición, el backend crea un grupo nuevo con el usuario de dueño y avisa al cliente. Invitación de miembros: - En el caso de ser invitado por el owner: Se incluyen al grupo de forma automática y se avisa a los clientes activos. - En otros casos: Se envía una 'request' al owner para

	que acepte al usuario, si se acepta, se le incluye de la forma mencionada anteriormente, sino se avisa al usuario de que no se pudo unir.

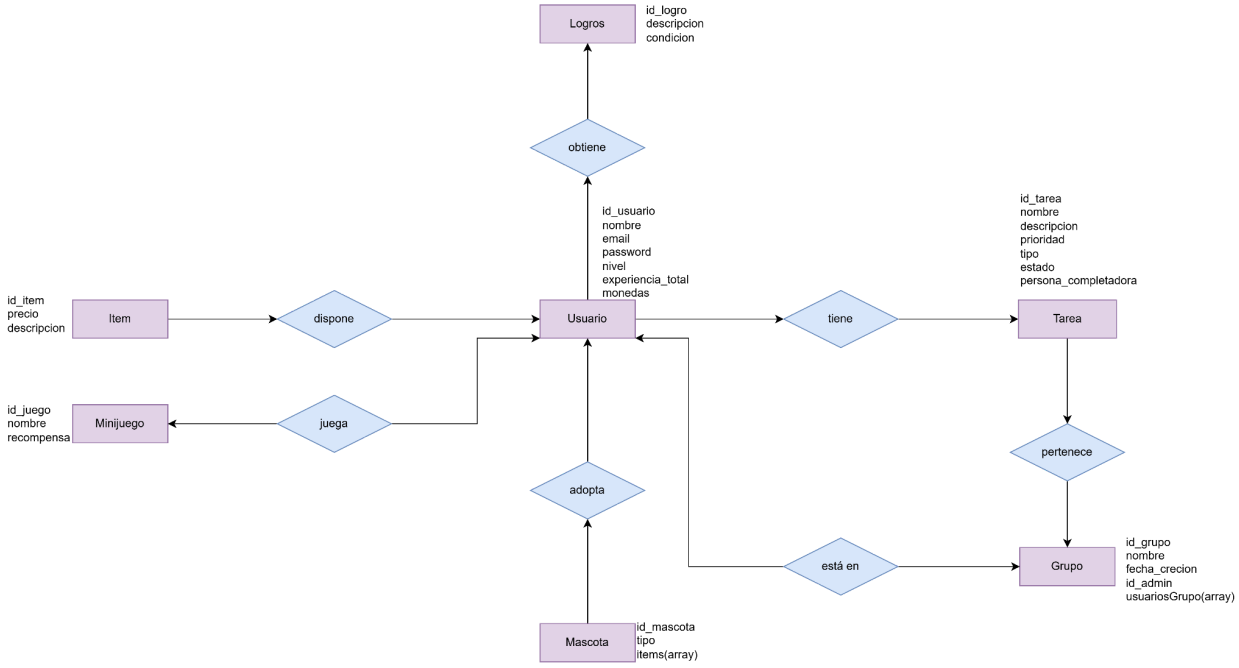
Caso 6: Minijuego con recompensa

Accionador	Usuario
Condiciones	No hay condiciones
Pasos a seguir	Juego: - Seleccionar el minijuego deseado, iniciar la sesión de juego, jugar hasta que termine el juego para obtener una puntuación. Backend: - Reciba la puntuación, haga los cálculos necesarios. - Envía la recompensa limitada al cliente. - Vigila el tiempo pasado en el minijuego para limitar la recompensa. - Valida y actualiza el balance del usuario.

Caso 7: Admin gestiona catálogo de tienda

Accionador	Administrador
Condiciones	Iniciar sesión como administrador
Pasos a seguir	- Se crean/editan ítems - Al guardar cambios, el servidor actualiza la tienda con esos cambios. - Al recibir cambios, los clientes activos recibirán el nuevo catálogo de forma automática gracias a un 'hook' o 'callback'.

7 - MODELO Y DISEÑO BD



7.1) Entidades principales

usuario: cuentas y rol
item: personalización (avatar base, mascota activa)
minijuego: relación amistad + estado
mascota: cosmético
tarea: tareas a realizar
grupo: grupo de amigos para realizar las tareas grupales
items: cosméticos de la tienda
logros: medalla otorgada bajo una condición.
minijuego: juegos con recompensas

8 - LÓGICA DE NEGOCIO MÍNIMA

- Se pide el input del usuario para algunos campos
- Se crea la tarea en la base de datos.
- Otorga XP y ludiones a los usuarios.
- Se evalúa si se consiguió un logro

- Se registra la tarea como completada en la base de datos.
- Comprueba si se tiene el dinero suficiente para hacer la transacción.
- Se hace una transacción donde se resta los ludiones al usuario y se le otorga el objeto.
- Se registra la transacción en la base de datos.
- Cada minijuego se valora por una puntuación, y los ludiones diarios de minijuego se calculan con esta puntuación.
- Se crea un grupo en la base de datos asignando al creador de líder
- Se genera un link de invitación.
- Al clicar en él, el usuario es metido dentro del grupo en la base de datos.
- El usuario indica los datos para registrarse.
- Se guarda el usuario en la base de datos.